

Homework 02

→ Objectives

To implement a *search* program in C program using system calls for files and directories.

→ Description

Find is a popular UNIX command that traverses a file hierarchy and performs various functions on each file in the hierarchy. The goal of this project is to implement a program similar called *search* that supports the following functionality:

- ☒ program – take directory name as input, print file hierarchy ...
 - The program should take the directory name from where to start the file traversal as a command-line argument and print the file hierarchy starting with the directory that is provided by the command-line argument.
- ☒ if the program is executed without any arguments ...
 - The program should print the file hierarchy starting with the current directory where the program is executed.
 - If there are no directories in the current directory only files are listed one per line.
- ☒ if there are other directories in the current directory ...
 - Then the directory name is first displayed on a separate line and then the files in that directory are listed one-per-line with one-tab indentation.
- ☒ if a file is a symbolic link ...
 - Then the program should display the symbolic link name and in parentheses the file name the link points to.
- ☒ the program should also support three command-line options:
 - 1) **-S**
This should list all files in the hierarchy and print the size, permissions, and last access time next to the filename in parenthesis. Print 0 for the size of a directory.
 - 2) **-s <file size in bytes>**
This should list all files in the hierarchy with file size less than or equal to the value specified.
 - 3) **-f <string pattern> <depth>**

This should list all files in the hierarchy that satisfy the following conditions:

- The file name contains the substring in the string pattern option
- The depth of the file relative to the starting directory of the traversal is less than or equal to the depth option. The starting directory itself has a depth of 0.

☒ the program should support not only each of these options separately ...

- But also any combination of these options. For examples:

- -S
- -s 1024
- -f jpg 1
- -S -s 1024
- -S -f jpg 2
- -s 1024 -f jpg 2
- -S -s 1024 -f jpg 1
- -S -f jpg 2 -s 1024

☒ if both -s and -f options are specified ...

- Then the program should list only those files that match both criteria. The order of the options should not matter.

☒ the program should support a fourth command-line option:

- **-t f**- list regular files only
- **-t d**- list directories only

→ Program Documentation and Testing

- 1) Use appropriate names for variables and functions
- 2) Use a Makefile to compile your program
- 3) Include meaningful comments to indicate various operations performed by the program.
- 4) Programs must include the following header information within comments:

```
/*  
Name:  
BlazerId:  
Project #:  
To compile: <instructions for compiling the program>  
To run: <instructions to run the program>  
*/
```

- 5) Test your program with the sample test cases provided as well as your own test cases.
- 6) You can include any comments you may have about testing in the README.txt file.

→ Submission Guidelines

- I) Use best software engineering practices to design and implement this homework.
- II) Use functions and organize these functions in multiple files if necessary.
- III) Use a Makefile to compile and build the multiple files.
- IV) Document your code and include instructions for compiling and executing the program in the README.txt file.
- V) Test your program and describe how you tested this program in the README.txt file.
- VI) To submit:
 - Upload all the source files and documentation
 - Make sure you have shared your GitHub repo with the TA